PDF output

Typeset every docs page as a PDF from the same document tree the web page comes from, with executed output included.

Shravan Goswami / https://shravangoswami.com

Almanac can typeset every docs page as a PDF during the build. Not a print stylesheet or a headless browser screenshot: a real typeset document, produced by Typst from the same tree the web page was rendered from.

> This site has it on. Every documentation page has a PDF link at the bottom, and
>
> the sidebar has one for the whole thing as a book.
>
> Executable code
>
> is the interesting page: its PDF holds the same computed Python and R output the page shows.

That last part is the point. The PDF and the page cannot disagree, because there is one parse, one set of resolved cross references, and one set of executed results feeding both.

Turning it on

```
almanac({
  title: "My Project",
  future: { myst: true, pdf: true },
})
```

`pdf` requires `myst`, and the build says so if you forget:

```
[almanac] Invalid configuration:
  - future.pdf: future.pdf needs future.myst: the PDF is built from the MyST tree
```

Install the toolchain, which is an optional peer like the rest:

```
npm install myst-to-typst @myriaddreamin/typst-ts-node-compiler
```

No system Typst installation is needed. The compiler is a native module with Typst built in.

What you get

One PDF per docs page, written to `dist/pdf/<page id>.pdf`, where the id is the same one the sidebar uses. So `docs/guides/writing.md` becomes `/pdf/guides/writing.pdf`.

Each page in the sidebar gains a PDF link next to “Edit this page”. The link appears only when the feature is on.

What survives into the document:

- **Headings, lists, tables, and quotes**, typeset rather than screenshotted.
- **Math**, properly set. Inline and display both.
- **Numbered figures with captions**, using the same numbers the page shows.
- **Cross references**, with the text they resolved to.
- **Admonitions**, as coloured blocks.
- **Executed output**, printed under the code that produced it.

Deliberate limitations

- **Remote images are not embedded.** Typst cannot fetch over the network, and a build that reaches out to load a figure is a build that fails in CI behind a firewall. A remote image becomes a visible `[image: url]` line, and the build logs which page it was on. Local images are embedded normally.
- **A page that fails to typeset does not fail the build.** It is reported with the compiler's own diagnostics and skipped. The HTML for that page was already correct, and losing the whole site because one document confused the typesetter is the wrong trade.
- **Only docs pages.** Blog posts are not typeset.
- **Only the default version and locale.** A variant page can share its source file with the page it inherits from, so a variant PDF would silently be a copy of another one. The link is hidden there rather than pointing at something wrong.

The whole thing as a book

One PDF holding every page, with a cover, a clickable table of contents, and numbered chapters:

```
almanac({
  pdf: {
    book: {
      enabled: true,
      filename: "handbook.pdf",
      subtitle: "The complete documentation",
      toc: { depth: 3 },
    },
  },
  future: { myst: true, pdf: true },
})
```

What you get in `dist/pdf/handbook.pdf`:

- **A cover page** with the title, subtitle, author, site, and build date.
- **A table of contents** with real page numbers and dot leaders, whose entries are clickable. It is Typst's own outline, so it cannot drift from the content.
- **Numbered headings**, `1`, `1.1`, `1.1.1`, restarting the page count after the front matter so page 1 is the first chapter rather than the cover.
- **PDF bookmarks**, so a reader's outline pane is populated without anything extra.

Chapter order follows your sidebar, because that is the reading order you already chose. Pages the sidebar does not mention still appear, sorted, after the ones it does; nothing is silently dropped. Override it explicitly if you want:

```
book: { enabled: true, order: ["index", "start/installation", "guides/writing"] }
```

Set `perPage: false` alongside it to publish only the book.

Customizing the output

Two levels. The structured options cover the common cases:

```
pdf: {
  paper: "us-letter",
  margin: "(x: 2cm, y: 2.5cm)",
  bodyFont: ["Georgia", "Liberation Serif"],
  monoFont: ["JetBrains Mono"],
  bodySize: "11pt",
  accent: "#b45309",
  numberHeadings: true,
}
```

`margin` is passed to Typst verbatim, so anything Typst accepts works there.

Beyond that, replace the template entirely. A template is a Typst file, and whatever it does is what the document does:

```
pdf: {
  template: "./pdf/page.typ",
  book: { enabled: true, template: "./pdf/book.typ" },
}
```

Almanac defines a Typst dictionary named `almanac` before your template runs, so the template has the metadata it needs:

Field	Contents
`almanac.title`	Page title, or the book title
`almanac.subtitle`	Page description, or the book subtitle
`almanac.author`	From `author.name`
`almanac.source`	Your `site` URL
`almanac.kind`	`"page"` or `"book"`
`almanac.date`	Build date, `YYYY-MM-DD`
`almanac.chapters`	Chapter titles in order, for a book

A complete per-page template:

```
// pdf/page.typ
#set page(paper: "a5", margin: 1.4cm, numbering: "1")
#set text(size: 9pt, font: ("DejaVu Sans",))
#set par(justify: true)
#show link: set text(fill: rgb("#b45309"))

#if almanac.title != "" [
  #block(fill: rgb("#f5f2e8"), inset: 10pt, width: 100%, radius: 3pt)[
    #text(size: 15pt, weight: "bold")[#almanac.title]
    #if almanac.subtitle != "" [
      \ #text(size: 8pt, fill: luma(90))[#almanac.subtitle]
    ]
  ]
  #v(0.8em)
]
```

A custom template replaces the styling and the title block, and nothing else. The handful of definitions `myst-to-typst` requires are always emitted, so a template can be three lines without failing to compile.

An empty template file is meaningful: it strips the built-in furniture and leaves Typst's defaults.

Speed

The Typst compiler is created once per build and reused across pages, because it carries a font book and a parsed standard library that cost real time to build. A site of twenty pages typesets in a couple of seconds after that.

Nothing about the PDF step is cached between builds. Unlike code execution, where skipping unchanged work is the whole feature, typesetting is fast enough that a cache would mostly add ways to be wrong.